Day 5 – Reactive Forms & Validation

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Dev Server Running**: Ensure `ng serve` is active in the terminal.
   - **IDE Ready**: Open your project in VS Code, ready to generate a new component and modify reactive form imports.

2. **Prerequisites**:
   - You must have completed Day 4 successfully (understanding routing layouts, components, and service access).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Contrast Angular's Template-driven forms with Reactive forms.
- Configure and import `ReactiveFormsModule` inside standalone components.
- Use `FormControl`, `FormGroup`, and `FormBuilder` to model form data structures.
- Apply built-in validators (`required`, `min`, `maxLength`) to form fields.
- Write a custom validation function to check specific business constraints.
- Display contextual validation error messages in the template layout based on control states (`touched`, `invalid`).

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Define form complexity. Contrast template binding with programmatic reactive form groups.
2. Key Concepts & Core Ideas	7 min	Explain Reactive form models, validations, control states, and custom validators.
3. Live Coding Walkthrough	23 min	Install forms modules, build reactive forms with FormBuilder, write custom validators, and display UI error messages.

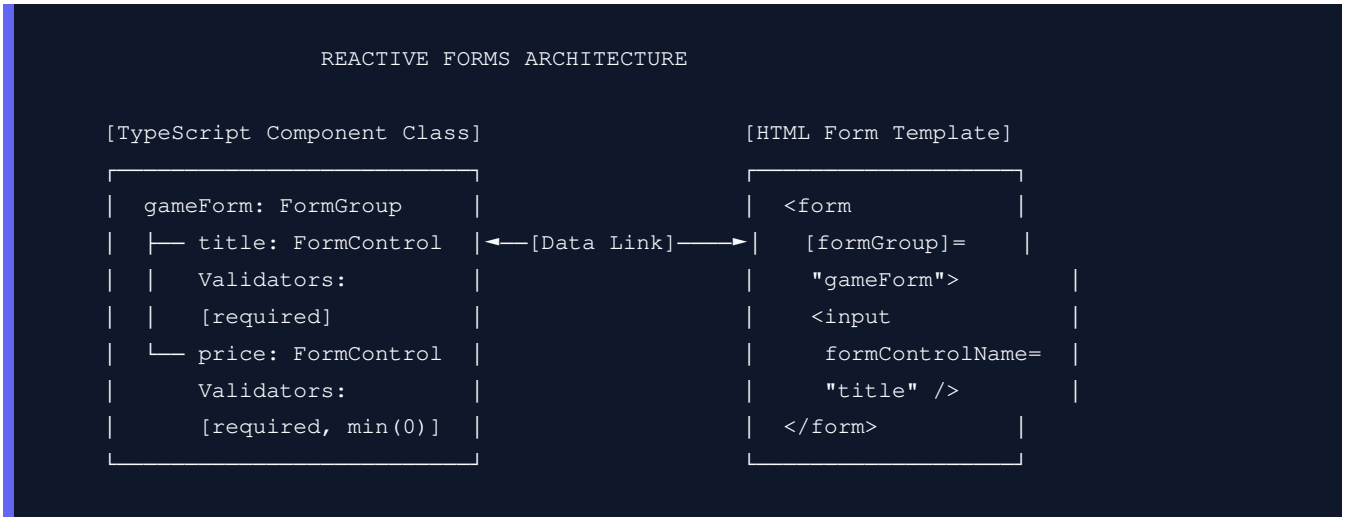
4. Check for Understanding	5 min	Q&A on validation states, dynamic status changes, and form submissions.
5. Class Wrap-up & Checkpoint	3 min	Verify validation blocks and error messages behave correctly in the UI.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

Forms are the primary source of user input in web applications. Without strong validation, users can submit garbage data (like negative prices or empty titles), crashing our database backend. Angular provides **Reactive Forms** which allow us to define the structure, initial state, and validation rules of our inputs programmatically in our TypeScript code, making it easy to test and validate.

- **Reactive Forms Flow**: Instead of letting the HTML template handle form bindings (which gets messy for complex validations), the TypeScript class acts as the single source of truth. The HTML input elements simply link to the controls we define in TypeScript.



2. Key Concepts & Core Ideas (7 min)

Introduce these reactive form concepts:

- **Reactive Forms vs Template-Driven**:
  - *Template-Driven*: Uses `[(ngModel)]` in HTML. Simple but hard to test or manage for complex forms.
  - *Reactive*: Programmatic control in TS. Explicit, testable, handles complex dynamic scenarios, and supports observable streams.
- **`FormControl`**: Tracks the value and validation status of an individual input field.
- **`FormGroup`**: Groups multiple `FormControl` objects together, aggregating their values and validation

statuses.

- **`FormBuilder`**: A helper service that provides syntactical sugar to construct form control groups quickly.
- **Validation States**: Controls track user interaction states to prevent showing error messages too early:
  - `pristine`: The user has not modified the input value yet.
  - `dirty`: The user has changed the input value.
  - `untouched`: The user has not visited (focused/blurred) the input box yet.
  - `touched`: The user has focused and blurred the input box.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Creating a Form Component

- **Concept**: Generate a standalone component called `add-game-form`.
- **Code**:

```
# Generate component
ng generate component components/add-game-form
```

- **Verify**: Check that the folder `src/app/components/add-game-form/` contains the generated files.

Step 3.2: Implementing Reactive Form Class Logic

- **Concept**: Open `src/app/components/add-game-form/add-game-form.component.ts`. We will define the form controls, attach built-in validators, write a custom validator to prevent lowercase titles, and inject `GameService` to save inputs.
- **Code**: Modify `src/app/components/add-game-form/add-game-form.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, FormGroup, Validators, ReactiveFormsModule, AbstractControl, Validatio
import { GameService } from '../../services/game.service';
import { Router, RouterModule } from '@angular/router';

@Component({
  selector: 'app-add-game-form',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule, RouterModule],
  templateUrl: './add-game-form.component.html',
  styleUrls: ['./add-game-form.component.css']
})
export class AddGameFormComponent implements OnInit {
  gameForm!: FormGroup;

  // Inject FormBuilder helper, GameService to save data, and Router to redirect
  constructor(
```

```
    private fb: FormBuilder,
    private gameService: GameService,
    private router: Router
  ) {}

  ngOnInit(): void {
    this.initForm();
  }

  # Initialize the form controls using FormBuilder
  initForm(): void {
    this.gameForm = this.fb.group({
      // title control: initial value empty string, validators: required, and custom uppercase
      title: ['', [Validators.required, this.titleUppercaseValidator]],
      // price control: initial value 0, validators: required and minimum value 1
      price: [0, [Validators.required, Validators.min(1)]]
    });
  }

  # Custom Validator: Checks if the first letter of the game title is capitalized
  # Returns null if valid, or a ValidationErrors object if invalid
  titleUppercaseValidator(control: AbstractControl): ValidationErrors | null {
    const value = control.value as string;
    if (!value) return null; // Let 'required' validator handle empty values

    const firstLetter = value.charAt(0);
    if (firstLetter !== firstLetter.toUpperCase()) {
      # Return error key 'titleLowercase' with true state
      return { 'titleLowercase': true };
    }
    return null;
  }

  # Getters to simplify HTML markup code checks
  get title() { return this.gameForm.get('title'); }
  get price() { return this.gameForm.get('price'); }

  # Form Submission Handler
  onSubmit(): void {
    if (this.gameForm.invalid) {
      // Mark all fields touched to trigger error styling display
      this.gameForm.markAllAsTouched();
      return;
    }

    const { title, price } = this.gameForm.value;
    this.gameService.addGame(title, price).subscribe({
      next: () => {
        // Redirect back to catalog inventory after successful insert
        this.router.navigate(['/games']);
      }
    });
  }
}
```

```
      <button type="submit" class="btn-submit">Add Game</button>
      <a routerLink="/games" class="btn-cancel">Cancel</a>
    </div>

  </form>
</div>
```

Step 3.4: Registering the Route

- **Concept**: Update `app.routes.ts` to add a path pointing to our new creation form component.
- **Code**: Modify `src/app/app.routes.ts`:

```
import { Routes } from '@angular/router';
import { GameListComponent } from './components/game-list/game-list.component';
import { GameDetailComponent } from './components/game-detail/game-detail.component';
import { AddGameFormComponent } from './components/add-game-form/add-game-form.component'; //

export const routes: Routes = [
  { path: '', redirectTo: 'games', pathMatch: 'full' },
  { path: 'games', component: GameListComponent },
  { path: 'games/add', component: AddGameFormComponent }, // Register before parameterized rou
  { path: 'games/:id', component: GameDetailComponent },
  { path: '**', redirectTo: 'games' }
];
```

Modify `src/app/app.component.html` to add a link to the form:

```
<header>
  <h1>Welcome to GameKey Store</h1>
  <nav>
    <a routerLink="/games" routerLinkActive="active-link">Catalog Inventory</a>
    <a routerLink="/games/add" routerLinkActive="active-link">Add Game</a> <!-- Add form link
  </nav>
</header>
<router-outlet></router-outlet>
```

- **Verify**: Check `http://localhost:4200/games/add`. Click "Add Game" immediately with empty inputs and verify the fields turn red and display "Title is required" and "Price must be at least $1.00". Type `test` (lowercase) and verify it catches the custom capitalization rule. Type `Portal 3` and `25` and submit; it should redirect to the catalog list showing the new record.

4. Check for Understanding (5 min)

> ***Question**: "Why did we register `{ path: 'games/add', ... }` before `{ path: 'games/:id', ... }` in the routes configuration?"*

> ***Answer***: *Router routes are matched sequentially in the order defined in the routes list. If `{ path: 'games/:id', ... }` were placed first, Angular Router would match `games/add`, parse `add` as the dynamic `:id` parameter, and load `GameDetailComponent` with ID `"add"`, which would fail. Placed first, `games/add` matches exactly.*

> ***Question***: *"What do the states `pristine` vs `dirty` and `touched` vs `untouched` track, and why are they useful?"*
> ***Answer***: `pristine/dirty` *track whether the user has modified the input value.* `touched/untouched` *track whether the user has clicked inside the field and then shifted focus away (blur event). They are useful because we don't want to show red error messages when a page first loads (which is pristine/untouched), only showing errors after the user interacts and leaves the field invalid.*

> ***Question***: *"How do you access the current aggregated JSON value of a FormGroup?"*
> ***Answer***: *You can access it using the `.value` property on the FormGroup instance (e.g. `this.gameForm.value`). This returns a standard Javascript object matching the control names and values: `{ title: 'Portal 3', price: 25 }`.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ The `AddGameFormComponent` uses `FormBuilder` to model inputs.
  - ○ Submitting invalid inputs triggers validation errors and blocks service dispatch.
  - ○ The custom uppercase validator works correctly.
  - ○ Valid form submission redirects to the catalog and displays the new item.